

Actividad Evaluativa 2 - Informe individual

Buscador y filtro de catálogo en tiempo real

CARRERA: Ingeniería en Sistemas de Información

MATERIA: Paradigmas y Lenguajes de Programación III

COMISIÓN: A

PROFESOR: Sicardi, Dante

ESTUDIANTE: Tarnoski, Santiago

FECHA: 29-09-2026

REPOSITORIO: <https://github.com/santiagotarnoski/gym-control-ae1>



INTRODUCCIÓN

Gym Control es un sitio de consulta de planes de gimnasio desarrollado en la Actividad Evaluativa 1. Esta Actividad Evaluativa 2 amplía el catálogo en tarjetas con un buscador y un filtro por duración. La solución conserva los cuatro planes existentes y permite localizar una opción antes de acceder al formulario de selección.

La tarjeta asignada es la opción 3, Buscador y filtro de catálogo en tiempo real. Exige registrar los eventos con `addEventListener`, cargar el catálogo completo mediante `fetch` y `async/await`, filtrar los datos en memoria con `Array.filter` y presentar los resultados en el DOM. La fundamentación de la Unidad 1 corresponde a SOHDM, aplicada al escenario de exploración y búsqueda.

Este informe individual documenta el aporte de Santiago Tarnoski: la implementación de la tarjeta 3, su fundamentación con SOHDM, las decisiones técnicas y las evidencias de verificación. Complementa el informe grupal, que reúne las tres tarjetas del equipo. El alcance es académico: los datos son de ejemplo, no hay persistencia ni contratación real.

Responsabilidad individual y organización del equipo

La responsabilidad asignada a Santiago Tarnoski corresponde al buscador y filtro del catálogo. Comprende la carga del JSON, el registro de eventos, el filtrado, la presentación de resultados, los estados de la interfaz y la navegación al formulario con el plan elegido. La fundamentación incluye el escenario E01 y su mapa navegacional SOHDM.

El informe grupal identifica a Franco Olexyn como responsable de la tarjeta 4, registro y autenticación de socios con VSDM, y a Santino Martins como responsable de la tarjeta 11, reclamos y consultas con fundamentación conceptual de RNA. Esos módulos forman parte del contexto de integración; este informe desarrolla y evalúa específicamente la tarjeta 3.

Alcance y trazabilidad

Requisito	Evidencia en la implementación
Continuidad con AE1	Los cuatro planes, precios, páginas y estilos de Gym Control permanecen.
Eventos desacoplados	<code>input</code> , <code>change</code> y <code>click</code> se registran en <code>js/app.js</code> con <code>addEventListener</code> .
HTTP y asincronía	<code>fetch</code> y <code>async/await</code> cargan <code>data/planes.json</code> desde un servidor local.
Búsqueda dinámica	<code>Array.filter</code> combina texto y duración; el DOM presenta tarjetas y contador.
Metodología SOHDM	Escenario E01, objetos y mapa navegacional de consulta y selección.



DESARROLLO

1 Descripción del módulo

El usuario accede a listado_box.html desde la portada o desde el listado en tabla. Antes de ofrecer la búsqueda, js/app.js consulta por HTTP el archivo data/planes.json. Cada registro describe el identificador, nombre, duración, precio, categoría, descripción y marca de destacado de un plan. Los precios se conservan del proyecto original: Mensual \$25.000, Trimestral \$67.500, Semestral \$120.000 y Anual \$216.000.

El campo de texto busca coincidencias en nombre, descripción y categoría. La selección de duración admite 1, 3, 6 y 12 meses. Ambos criterios se aplican en conjunto. El usuario puede cambiar el texto sin pulsar un botón, limpiar los filtros o elegir un resultado. Por ejemplo, escribir renovacion encuentra renovación sin exigir la tilde. El término mensual devuelve también el Plan Anual porque su descripción menciona valor mensual.

2 Organización técnica

Componente	Responsabilidad
listado_box.html	Presenta controles, estado de carga, contenedor y mensaje sin resultados.
data/planes.json	Almacena los cuatro planes de ejemplo de la AE1.
js/app.js	Carga, valida, filtra, crea tarjetas y conserva el plan al navegar.
css/styles.css	Mantiene el diseño y adapta los controles a la vista móvil.
comprar.html	Recibe el identificador del plan y conserva las funciones previas.

La carga usa fetch("data/planes.json") con await. El programa verifica response.ok y valida el formato antes de habilitar los filtros. Un error HTTP, de red o de datos muestra un mensaje y un botón Reintentar. El catálogo vacío tiene un mensaje diferente del caso sin coincidencias. Al modificar los criterios, el programa aplica Array.filter sobre los registros ya cargados, sin realizar otra petición [2, 3].

Las tarjetas se construyen con createElement y textContent, y se insertan con replaceChildren. Esta decisión permite presentar el contenido JSON como texto. Los archivos originales js/subtotal.js y js/cupon.js no se modificaron; al seguir un resultado, un parámetro plan preselecciona el radio correspondiente y dispara su evento change para actualizar el subtotal existente.



3 Escenario E01 de exploración y búsqueda

Actor principal: visitante o socio interesado en comparar planes. Precondición: el sitio se sirve mediante HTTP y el catálogo puede cargarse. Objetivo: encontrar un plan compatible con los criterios del usuario y continuar a su selección.

Paso	Acción y respuesta esperada
1	Abrir Planes en cards; el sistema carga y presenta cuatro tarjetas.
2	Escribir progreso; aparece el Plan Semestral por su categoría.
3	Elegir 6 meses; permanece el resultado compatible.
4	Elegir el plan; se abre comprar.html?plan=semestral.
5	Consultar el formulario; el plan aparece marcado y el subtotal se calcula.

Flujos alternativos: si los filtros no encuentran registros, se muestra cero resultados y se permite modificar o limpiar la consulta. Si falla la carga, se bloquean los controles y se ofrece reintento. Si el JSON está vacío, se indica que no hay planes disponibles. La postcondición del flujo principal es una selección visual de plan; no se asigna una membresía.

4 Fundamentación de SOHDM

SOHDM (Scenario Based Object Oriented Hypermedia Design Methodology) usa escenarios para identificar requisitos de sistemas hipertexto y emplea vistas de objetos para definir unidades navegacionales. Lee, Lee y Yoo describen seis fases: análisis del dominio, modelado de objetos, diseño de vistas, diseño de navegación, diseño de implementación y construcción [1].

Fase	Aplicación en Gym Control
Análisis del dominio	Actor, objetivo, flujo principal y alternativas del escenario E01.
Modelado de objetos	Plan, Consulta y Catálogo como objetos conceptuales.
Diseño de vistas	Barra de filtros, tarjetas, contador y estados de error o vacío.
Diseño de navegación	Accesos desde Inicio y tabla; salida a detalle o selección.
Diseño de implementación	JSON, funciones JavaScript, estructura HTML y estilos.
Construcción	Integración del código y pruebas de navegador con servidor HTTP.

Plan aporta los atributos que se buscan y presentan. Consulta representa el texto normalizado y la duración. Catálogo contiene los planes cargados y obtiene las coincidencias. Las vistas muestran únicamente la información necesaria para explorar y decidir. La aplicación usa registros JSON y funciones JavaScript; no implementa un motor formal de SOHDM.

5 Mapa navegacional

El mapa deriva del escenario E01. La navegación existente Inicio → Listado en tabla o Catálogo en tarjetas se mantiene. Búsqueda y resultados son estados de la página listado_box.html, no páginas independientes. Desde una tarjeta, el usuario puede ir a la selección con el ID del plan. La ficha producto.html del proyecto previo corresponde específicamente al Plan Mensual.

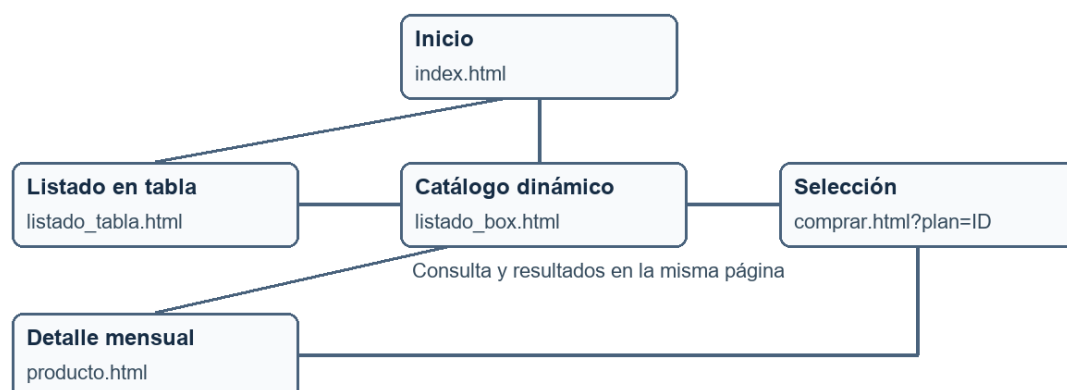


Figura 1 Mapa navegacional del escenario de exploración y búsqueda

Los enlaces de retorno permiten revisar la tabla, modificar filtros o limpiar la consulta. Si falla la carga, Reintentar vuelve a ejecutar la petición HTTP sobre el mismo nodo. El archivo editable docs/mapa-navegacional-tarjeta3.mmd conserva el diagrama.

6 Interacción y estados de la interfaz

El evento input del campo de búsqueda y change del selector de duración llaman a filtrarPlanes(). El botón Limpiar filtros restaura ambos controles y devuelve el foco al campo de texto. El botón Reintentar invoca cargarCatalogo(). Todos se vinculan mediante addEventListener en js/app.js; el HTML no contiene atributos onclick ni onsubmit [4].

El contador informa cuántos planes se muestran de los cuatro disponibles. role="status" y aria-live="polite" comunican los resultados; aria-busy indica la carga. Mientras la respuesta no es válida, los controles permanecen deshabilitados. El CSS reorganiza los controles y las tarjetas en una columna para teléfonos.

7 Verificación

Se ejecutaron 36 comprobaciones en Google Chrome mediante un servidor HTTP local. El registro completo se conserva en docs/pruebas-tarjeta3.json y el procedimiento reproducible en tests/catalogo.cjs. Las pruebas cubren la carga de las cuatro tarjetas, búsquedas con tildes y mayúsculas, combinación de filtros, cero resultados, limpieza, selección de plan, compatibilidad con subtotal y cupón, vista móvil y fallos HTTP, de red o de JSON.



Caso representativo	Resultado observado
Búsqueda por progreso y 6 meses	Solo se presenta el Plan Semestral.
Búsqueda incompatible	Se informa 0 de 4 planes y se ofrece limpiar.
Plan Trimestral elegido	comprar.html preselecciona el plan y muestra \$67.500.
Error HTTP 404	Los filtros se bloquean; el reintento recupera el catálogo.
Vista de 390 píxeles	No existe desbordamiento horizontal.

El resultado global fue de 36 verificaciones correctas. El contenido del JSON se insertó como texto, y el recorrido normal no produjo errores JavaScript. Las capturas de pantalla se incluyen en Anexos.

8 Decisiones técnicas y relación con el trabajo grupal

Una decisión central fue separar la carga del catálogo de la búsqueda. La petición HTTP ocurre al iniciar la página o al reintentar una carga fallida; cada cambio de texto o duración utiliza el arreglo que ya está en memoria. Esto evita peticiones repetidas y permite actualizar los resultados inmediatamente.

El resultado se conserva al navegar mediante el identificador del plan en la URL. Esa integración aprovecha el formulario existente y su cálculo de subtotal. Los estados de carga, error, catálogo vacío y ausencia de coincidencias se distinguen para orientar al usuario y evitar que filtre datos que todavía no están disponibles.

En la versión grupal, los bloques de las tarjetas 3, 4 y 11 conviven en `js/app.js`. El catálogo sigue siendo público para el visitante; el acceso del socio y el formulario de reclamos amplían el recorrido sin sustituir la búsqueda. La comprobación de cada módulo se documenta por separado, por lo que las 35 verificaciones de acceso y las 12 de reclamos no se presentan aquí como resultados de la tarjeta 3.

La actividad vincula los requisitos del escenario con los eventos y estados concretos de una página web. Los principales conceptos aplicados en este aporte son asincronía, validación de una respuesta HTTP, operaciones sobre arreglos y actualización del DOM. Como mejora futura, el catálogo podría consumirse desde un servicio con persistencia; esa ampliación requeriría revisar el contrato de datos y sus rutas de error.



CONCLUSIONES FINALES

El aporte individual de Santiago Tarnoski incorpora a Gym Control el módulo de la tarjeta 3, distinto del subtotal, del cupón y de las tarjetas 4 y 11 del equipo. Cumple la tarjeta 3 mediante eventos desacoplados, una carga HTTP asíncrona del catálogo, filtrado de `Array.filter` en memoria y actualización inmediata del DOM.

El escenario E01 permitió justificar los campos de consulta, los estados de resultados y los accesos entre catálogo y formulario. El mapa navegacional SOHDM relaciona esa interacción con las pantallas existentes y conserva la continuidad del proyecto. Las pruebas confirman los casos esperados y las rutas de error del prototipo.

El catálogo JSON conserva datos comerciales de ejemplo. Para operar membresías reales sería necesario un servicio de servidor, persistencia y validación de las operaciones. Esta entrega se limita a la consulta, exploración y selección visual de planes.

BIBLIOGRAFÍA

- [1] Lee, H., Lee, C., & Yoo, C. (1999). A scenario-based object-oriented hypermedia design methodology. *Information & Management*, 36(3), 121-138.
[https://doi.org/10.1016/S0378-7206\(99\)00011-7](https://doi.org/10.1016/S0378-7206(99)00011-7)
- [2] MDN Web Docs. (s. f.). Using the Fetch API.
https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
- [3] MDN Web Docs. (s. f.). `Array.prototype.filter()`.
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array/filter
- [4] MDN Web Docs. (s. f.). `EventTarget.addEventListener()`.
<https://developer.mozilla.org/es/docs/Web/API/EventTarget/addEventListener>
- [5] Tarnoski, S., Martins, S., & Olexyn, F. (2026). Gym Control: análisis y diseño de la Actividad Evaluativa 1. Repositorio del proyecto. <https://github.com/santiagotarnoski/gym-control-ae1>
- [6] Martins, S., Olexyn, F., & Tarnoski, S. (2026). Actividad Evaluativa 2: integración JS y metodologías web en Gym Control [Informe grupal]. Repositorio del proyecto.
<https://github.com/santiagotarnoski/gym-control-ae1>



ANEXOS

Anexo A Evidencia de búsqueda

Buscá el plan que se adapte a vos

Buscar por nombre, descripción o categoría

compromiso



Duración del plan

12 meses

1 de 4 planes encontrados.

Compromiso

Plan Anual
12 meses

\$ 216.000
/ total

La opción de mayor duración y mejor valor mensual.

Elegir plan

Figura 2 Búsqueda por categoría y duración con un resultado

Anexo B Estado sin resultados

Buscá el plan que se adapte a vos

Buscar por nombre, descripción o categoría

compromiso



Duración del plan

1 mes

0 de 4 planes encontrados.

No encontramos planes con esos filtros. Probá otra búsqueda o limpiá los filtros.

Figura 3 Filtros incompatibles y mensaje al usuario